Your Name: Caleb Zhong

Your Andrew ID: yuzhong4

Homework 1

0 Introduction

0.1 Collaboration and Originality

1. Did you receive help of any kind from anyone (other than the instructor or TAs) in developing your software for this assignment (Yes or No)? If you answered Yes, provide the name(s) of anyone who provided help, and describe the type of help that you received. No

2. Did you give help of any kind to anyone in developing their software for this assignment (Yes or No)? If you answered Yes, provide the name(s) of anyone that you helped, and describe the type of help that you provided. No

3. Did you examine anyone else's software for this assignment (Yes or No)? Do not describe software provided by the instructor. No

4. Are you (or the course instructor) the author of every line of source code submitted for this assignment (Yes or No)? If you answered No: Yes
    a. identify the software that you did not write,
    b. explain where it came from, and
    c. explain why you used it.

5. Are you the author of every word of your report (Yes or No)? If you answered No: Yes
    a. identify the text that you did not write,
    b. explain where it came from, and
    c. explain why you used it.

1 Structure in Boolean Queries

1.1 Structure in Boolean Queries: Queries

1.1.1 General Strategy for Manual Structured Queries

For manual structured queries, I adopt a concept-centered and rule-based strategy. For each information need, I first identify the core semantic concepts and enforce their co-occurrence by using operators such as NEAR or WINDOW to keep the phrase-level meaning. What's more, additional context or user intent is incorporated as soft constraints using operator OR. Since this operator provides softer filters, which can avoid overly restricting recall. Last but not least, I also applied field restrictions selectively when they provide clear semantic benefits such as authoritative entities in title, event-related signals in inlink.

In this experiment, I applied the same structural principles to both short and long queries to keep the core concept consistently. For extension, long quries added some intent-related constraints based on the short one.

1.1.2 Show the Structured Queries

1. Q121: Sit and Reach Test

   Short query: `#WINDOW/10( sit reach test )`

   Long query: `#AND( #NEAR/3(sit reach test) #OR( #NEAR/3( physical rehabilitation ) treatment.body ) )`

Explanation: The short query treats “sit and reach test” as a compact phrase using a window operator to ensure tight semantic matching. The long query preserves this core structure and adds rehabilitation-related terms as soft constraints to make sure the test is used in physical therapy contexts.

2. Q129: Iowa Food Stamp Program

   Short query: `#AND( iowa #SYN(food stamp) program )`

   Long query: `#AND( iowa #SYN( food stamp assistance ) #OR( apply application program assistance.keywords ) )`

Explanation: The short query constrains the search to Iowa and expands “food stamp” via synonyms to capture common variations of the program name. The long query introduces application-related terms as weak constraints to retrieve guidance pages and softly reinforced them through the keywords field, since they often appears in service-oriented content.

3. Q133: All Men Are Created Equal

   Short query: `#WINDOW/12( all men created equal )`

   Long query: `#AND( #WINDOW/12( all men created equal ) #OR( jefferson said quote ) )`

Explanation: The short query is modeled as a single semantic unit using a window operator to ensure tight semantic matching. In the long query, attribution-related terms are added as soft constraints to encourage documents discussing authorship, since this sentence is believed said by Thomas Jefferson.

4. Q137: Rock and Gem Shows

Short query: `#AND( #NEAR/3( rock gem ) #OR( shows shows.inlink ) )`

Long query: `#AND( #NEAR/3( rock gem ) #OR( shows shows.inlink ) #OR( locations dates ) )`

Explanation: The short query uses proximity constraint ensures that “rock” and “gem” function as a unified event theme. The use of the inlink field as an optional signal exploits anchor-text descriptions of shows. The long query adds location and date terms to emphasize schedule-oriented results.

5. Q141: Virginia DMV Registration

Short query: `#AND( #SYN(va virginia ).title #SYN( dmv "department of motor vehicles" ).title registration )`

Long query: `#AND( #SYN(va virginia ).title #SYN( dmv "department of motor vehicles" ).title #OR(offices hours location) )`

Explanation: Both queries constrain authoritative entity names to the title field to improve precision for official DMV pages. Moreover, SYN operator is used to explicitly model common abbreviations and their expanded forms (e.g., “VA” vs. “Virginia”). The long query relaxes the service intent into soft constraints, allowing retrieval of office and hours information even when registration is not explicitly mentioned.

6. Q147: Tangible Personal Property Tax

Short query: `#WINDOW/15( tangible personal property tax )`

Long query: `#AND( #NEAR/3( tangible personal property ) #OR( tax definition purposes definition.keywords ) )`

Explanation: The short query treats the legal term as a fixed phrase to focus on relevant tax-related documents. The long query separates the core legal concept from definition-related terms, using OR to bias results toward explanatory content.

7. Q155: Last Supper Painting

Short query: `#AND( #NEAR/3( last supper) #OR( painting image picture) )`

Long query: `#AND( #NEAR/3( last supper ) #OR( leonardo "da vinci" ) #OR( painting image picture image.url ) )`

Explanation: The artwork title is enforced through proximity matching to preserve phrase-level meaning. Visual descriptors and author information are added as soft constraints to increase the likelihood of

retrieving image-oriented and authoritative art pages. Furthermore, terms are optionally matched in the URL field to favor image or gallery pages.

8. Q165: Blue-Throated Hummingbird

Short query: `#NEAR/3( blue throated hummingbird )`

Long query: `#AND( #NEAR/3( blue throated hummingbird ) #OR( migration patterns ) )`

Explanation: The species name is modeled as a single semantic unit to avoid partial biological matches. The long query adds migration-related terms as contextual match to get results more precisely.

9. Old Coins

Short query: `#AND( #OR( old ancient ) coins.inlink )`

Long query: `#AND( #OR( old ancient ) #OR( coins coins.inlink ) #OR( collecting begin guide ) )`

Explanation: Synonymous descriptors of age and designated items combined with inlink signals to emphasize the oriented resources. The long query introduces collecting-related terms as weak constraints to retrieve beginner guides without excluding general coin information pages.

10. Q189: General Schedule Pay Rate

Short query: `#AND( #SYN(gs "general schedule" ) #NEAR/3( pay rate ) )`

Long query: `#AND( #SYN(gs "general schedule" ) #OR( pay rate tables tables.url ) )`

Explanation: Synonym expansion aligns the abbreviation “GS” with its full form to ensure consistent matching. The short query tightly binds pay and rate, while the long query relaxes this constraint to capture official salary tables and explanatory documents. Meanwhile, table-related terms are optionally matched in the URL field, where official salary tables are commonly indicated.

11. Q191: Churchill Downs

Short query: `#NEAR/3( churchill downs )`

Long query: `#AND( #NEAR/3( churchill downs ) #OR( racing schedule ) )`

Explanation: The venue name is enforced as a phrase to avoid unrelated uses of “downs.” The long query adds schedule-related terms to bias retrieval toward race calendars rather than historical or news-oriented content.

12. Q193: Dog Clean Up Bags

Short query: `#WINDOW/12( dog clean up bags )`

Long query: `#AND( #WINDOW/12( dog clean up bags ).body #OR( order online ) )`

Explanation: The product name is treated as a compact phrase to reduce noise. The long query adds online ordering terms as soft constraints to favor purchasing-oriented content over general pet care discussions.

1.2 Structure in Boolean Queries: Experimental Results

	Unranked Boolean					
	Short Queries			Long Queries		
	BOW #AND (Exp-1.1a)	BOW #NEAR/3 (Exp-1.1b)	Structured (Exp-1.1c)	BOW #AND (Exp-1.1d)	BOW #NEAR/3 (Exp-1.1e)	Structured (Exp-1.1f)
MRR	0.1020	0.5033	0.2306	0.1524	0.0000	0.3525
P@10	0.0500	0.3000	0.1417	0.1000	0.0000	0.2000
P@20	0.0625	0.2542	0.1542	0.0958	0.0000	0.1833
P@30	0.0556	0.2278	0.1472	0.0806	0.0000	0.1556
MAP@1K	0.0834	0.1650	0.1293	0.0236	0.0000	0.1151
NDCG@10	0.2847	0.2389	0.1351	0.0378	0.0000	0.1883
NDCG@20	0.0387	0.2403	0.1450	0.0411	0.0000	0.1844
NDCG@30	0.0437	0.2602	0.1587	0.0378	0.0000	0.1774
R@100	0.2847	0.4307	0.4635	0.0624	0.0000	0.2653
R@1000	0.8410	0.5221	0.7692	0.0902	0.0000	0.4075
Run Time	00:08	00:08	00:11	00:26	00:26	00:16

	Ranked Boolean					
	Short Queries			Long Queries		
	BOW #AND (Exp-1.2a)	BOW #NEAR/3 (Exp-1.2b)	Structured (Exp-1.2c)	BOW #AND (Exp-1.2d)	BOW #NEAR/3 (Exp-1.2e)	Structured (Exp-1.2f)
MRR	0.3514	0.3847	0.5340	0.1086	0.0000	0.5007
P@10	0.2750	0.3500	0.3750	0.0917	0.0000	0.3333
P@20	0.2667	0.3333	0.3417	0.0917	0.0000	0.2917
P@30	0.2556	0.2833	0.3000	0.0889	0.0000	0.2472
MAP@1K	0.1944	0.1651	0.2138	0.0236	0.0000	0.1787
NDCG@10	0.1874	0.2690	0.3161	0.0324	0.0000	0.3118
NDCG@20	0.2099	0.3003	0.3194	0.0405	0.0000	0.3004
NDCG@30	0.2203	0.2993	0.3113	0.0424	0.0000	0.2761
R@100	0.5883	0.4526	0.6012	0.0770	0.0000	0.3940
R@1000	0.9388	0.5221	0.7692	0.1004	0.0000	0.5562
Run Time	00:09	00:08	00:11	00:27	00:26	00:16

1.3 Structure in Boolean Queries: Analysis

From the experimental results we can find that our manual queries have received high marks across a range of metrics. However, structure is powerful, but only when it reflects an understanding of the underlying information need. Simply adding operators or constraints does not guarantee improvement, and in several cases overly strict structure actually hurts performance.

For short queries, simple proximity constraints often provide large gains with minimal risk. For example, for Q165: "blue throated hummingbird", the short structured query `#NEAR/3( blue throated hummingbird )` works well because it enforces a tight semantic unit without adding unnecessary constraints. In contrast, fully structured queries that introduce additional intent terms or fields do not always outperform simple #NEAR queries for short inputs, suggesting that when the query already contains a clear core concept, additional structure may offer diminishing returns.

When constructing the queries, I also used a small trick which highly improve the performance. This particularly effective technique is introducing inferred but unstated query elements when they are strongly implied by the information need. For example, in Q133: "Who said all men are created equal", the structured long query `#AND( #WINDOW/12( all men created equal ) #OR( jefferson said quote ) )` adds "Jefferson" as an optional element, even though it does not appear in the original query. This small trick significantly improves accuracy by encoding background knowledge about the quote's authorship, while keeping it optional avoids excluding relevant explanatory pages. This illustrates how structured queries can benefit from reasoning about intent rather than relying strictly on surface terms.

Moreover, the use of document fields also plays an important role, but only when applied carefully. For instance, in Q189: "Find the General Schedule pay tables", the structured query `#AND( #SYN( gs "general schedule" ) #OR( pay rate tables tables.url ) )` uses the URL field as a soft signal. Many authoritative GS pay tables are hosted on government pages where "tables" appears directly in the URL. Adding this field constraint helps surface these documents earlier in the ranking without significantly harming recall. In contrast, making the URL constraint mandatory would have excluded many otherwise relevant pages, highlighting the importance of using fields as ranking hints rather than hard filters.

Comparing Unranked Boolean and Ranked Boolean, structure is clearly more valuable in the ranked setting. In Unranked Boolean retrieval, structure primarily affects whether documents are retrieved at all, making it easy to lose recall when constraints are too strict. However Ranked Boolean retrieval can exploit structured signal, such as NEAR, SYN, and field to better order results. This explains why structured queries consistently lead to larger gains in MRR, MAP, and NDCG under Ranked Boolean than under Unranked Boolean.

2 Document Representations

2.1 Document Representations: Parameters

Ranking length (Output length for all experiments) : 1000 documents

BM25 parameters: $k_1 = 1.2$, $b = 0.75$

Unigram weight: 0.7

Ordered bigram (#NEAR/1) weight: 0.2

Unordered bigram (#WINDOW/8) weight: 0.1

2.2 Document Representations: Experimental Results

	Ranked Boolean				
	Short Queries			Long Queries	
	body (Exp-1.2a)	title (Exp-2.1b)		body (Exp-1.2d)	title (Exp-2.1e)
MRR	0.3514	0.4841		0.1086	0.0000
P@10	0.2750	0.2000		0.0917	0.0000
P@20	0.2667	0.1375		0.0917	0.0000
P@30	0.2556	0.1083		0.0889	0.0000
MAP@1K	0.1944	0.0841		0.0236	0.0000
NDCG@10	0.1874	0.2203		0.0324	0.0000
NDCG@20	0.2099	0.1876		0.0405	0.0000
NDCG@30	0.2203	0.1811		0.0424	0.0000
R@100	0.5883	0.1729		0.0770	0.0000
R@1000	0.9388	0.1729		0.1004	0.0000
Run Time	00:09	00:01		00:27	00:01

	BM25					
	Short Queries			Long Queries		
	body (Exp-2.2a)	title (Exp-2.2b)	SDM (Exp-2.2c)	body (Exp-2.2d)	title (Exp-2.2e)	SDM (Exp-2.2f)
MRR	0.5178	0.6367	0.6800	0.4253	0.7066	0.6409
P@10	0.2833	0.3667	0.3667	0.3750	0.3750	0.4250
P@20	0.2958	0.2583	0.3542	0.3333	0.2708	0.3792
P@30	0.2806	0.2250	0.3250	0.3028	0.2444	0.3528
MAP@1K	0.2739	0.2318	0.3265	0.2750	0.2455	0.3575
NDCG@10	0.2503	0.3423	0.3038	0.3195	0.3809	0.4143
NDCG@20	0.2636	0.3151	0.3296	0.3230	0.3469	0.4258
NDCG@30	0.2784	0.3123	0.3425	0.3339	0.3549	0.4340
R@100	0.7401	0.4374	0.7265	0.6299	0.5034	0.6720
R@1000	0.9802	0.6328	0.9825	0.9593	0.6709	0.9808
Run Time	02:07	00:03	03:35	06:57	00:09	17:30

The Exp-1.2a and Exp-1.2d results from Section 1.2 are used as baselines for comparison.

2.3 Document Representations: Analysis

These contract experiment show that the effectiveness of document fields mostly depends on both query length and the retrieval model. Rather than one field being universally better, `title` and `body` play very different roles in retrieval.

From my perspective, I find the most noticeable difference is that `title` is precise but narrow, while `body` is broad but noisy. More specifically, `title` usually contain the main topic or entity of a document, so when a query matches the title, the document is often highly relevant. This explains why for short queries, title-based retrieval often achieves higher early precision and MRR. However, since titles are short, they cover fewer variations of an information need. As a result, title-based retrieval tends to miss many relevant documents, leading to much lower recall than body-based retrieval. In contrast, the body field contains explanations, paraphrases, and more background information, which allows it to retrieve a much larger set of relevant documents, even though ranking score may be a little bit lower.

Moreover, query length further amplifies this difference. We can see from the experimental results, for long queries, using the `title` field with Ranked Boolean performs extremely poorly (we can't even find a matched document). Since Ranked Boolean uses strict matching, when we combine long queries with the short title field, it overly constrains retrieval and results in no matched documents at all. In comparison, Body-based Ranked Boolean still retrieves some relevant documents in this setting, showing that `body` is much more robust to query verbosity. This elaborates that `title` works well only when the query is short and focused, while `body` always performs relatively well and robust .

The behavior of fields also depends heavily on the retrieval model. Ranked Boolean is very sensitive to field choice and query length. Small changes in field or query structure can lead to large performance drops. On the other hand, BM25 uses fields more effectively. Even though titles are short, BM25's soft matching and term weighting allow it to catch the concentrated information in titles while avoiding overly restrictive retrieval . As a result, title-based BM25 performs well for both short and long queries in terms of early ranking quality, while body-based BM25 continues to provide strong recall. This shows that BM25 is generally better at adapting to the characteristics of different fields.

Finally, when we focus on Exp-2.2c and Exp-2.2f, we will find that SDM further improves retrieval effectiveness, especially for long queries. By modeling term dependencies in the body text, this model helps reduce the impact of noise and capture phrase-level meaning, which achieve a vital improvement across the board. However, SDM introduces a significant runtime cost, particularly for long queries, explicitly showing an effectiveness–efficiency trade-off.

3 Multifield BM25

3.1 Multifield BM25: Parameters

Ranking length (Output length for all experiments) : 1000 documents

BM25 parameters: $k_1 = 1.2$, $b = 0.75$

Exp-3.1/2b Field Weights

body: 0.7 title: 0.3 url : 0.0 keywords: 0.0 inlink : 0.0

This is the strongest structure I construct for multifield BM25. I expect `body` to carry most of the semantic content, so it gets the largest weight. In order to get a improvement based on the baseline (100% `body`) , `title` is given a meaningful but smaller weight because it is usually concise and highly topical, which can improve early precision without sacrificing too much recall.

Exp-3.1/2c Field Weights

body: 0.4 title: 0.3 url : 0.1 keywords: 0.1 inlink : 0.1

This is a balanced setting which contains all of the fields. The idea is to keep `body`/`title` as the main signal while giving small, equal weights to `url`/`keywords`/`inlink` to test whether light metadata evidence can help ranking without letting these noisier fields dominate.

Exp-3.1/2d Field Weights

body: 0.3 title: 0.4 url : 0.2 keywords: 0.1 inlink : 0.0

This is a title-forward setting. Compared to 3.1/2c, I shift more weight from `body` to `title` and slightly increase `url`. The motivation is that many web pages have informative titles and structured URLs, so emphasizing them might boost precision. More importantly, I just want to reduce the impact of `body` continuously and see the score changes across all metrics.

Exp-3.1/2e Field Weights

body: 0.1 title: 0.1 url : 0.25 keywords: 0.45 inlink : 0.1

This is an intentionally aggressive metadata-heavy configuration. I overweight `keywords` and `url` to see what happens if we trust these fields too much. I kept small but non-zero title/body weights so the query is not completely disconnected from content, but the main goal is to observe how performance changes when metadata dominates.

3.2 Multifield BM25: Experimental Results

	Short Queries				
	body (Exp-2.2a)	**0.00 inlink 0.00 url 0.00 keywords 0.30 title 0.70 body (Exp-3.1b)**	**0.10 inlink 0.10 url 0.10 keywords 0.30 title 0.40 body (Exp-3.1c)**	**0.00 inlink 0.20 url 0.10 keywords 0.40 title 0.30 body (Exp-3.1d)**	**0.10 inlink 0.25 url 0.45 keywords 0.10 title 0.10 body (Exp-3.1e)**
MRR	0.5178	0.6806	0.5484	0.4711	0.4888
P@10	0.2833	0.4000	0.3750	0.3083	0.2917
P@20	0.2958	0.3167	0.3083	0.2750	0.2375
P@30	0.2806	0.2667	0.2583	0.2639	0.2083
MAP@1K	0.2739	0.3439	0.2759	0.2387	0.1540
NDCG@10	0.2503	0.3614	0.3209	0.2663	0.2370
NDCG@20	0.2636	0.3495	0.3174	0.2791	0.2318
NDCG@30	0.2784	0.3481	0.3153	0.3054	0.2307
R@100	0.7401	0.6851	0.6285	0.5555	0.3566
R@1000	0.9802	0.9614	0.9242	0.8839	0.7384
Run Time	02:07	02:24	02:57	02:51	03:08

	Long Queries				
	body (Exp-2.2d)	**0.00 inlink 0.00 url 0.00 keywords 0.30 title 0.70 body (Exp-3.2b)**	**0.10 inlink 0.10 url 0.10 keywords 0.30 title 0.40 body (Exp-3.2c)**	**0.00 inlink 0.20 url 0.10 keywords 0.40 title 0.30 body (Exp-3.2d)**	**0.10 inlink 0.25 url 0.45 keywords 0.10 title 0.10 body (Exp-3.2e)**
MRR	0.4253	0.8236	0.8020	0.7051	0.6389
P@10	0.3750	0.4750	0.4250	0.3750	0.3000
P@20	0.3333	0.3917	0.3500	0.3208	0.2250
P@30	0.3028	0.3361	0.2778	0.2778	0.1944
MAP@1K	0.2750	0.3828	0.3231	0.3032	0.1513
NDCG@10	0.3195	0.4683	0.4060	0.3744	0.2801
NDCG@20	0.3230	0.4627	0.3925	0.3668	0.2540
NDCG@30	0.3339	0.4693	0.3777	0.3767	0.2432
R@100	0.6299	0.6618	0.6560	0.6311	0.3603
R@1000	0.9593	0.9563	0.9231	0.8540	0.6894
Run Time	06:57	07:55	10:30	10:01	11:20

The Exp-2.2a and Exp-2.2d results from Section 2.2 are used as baselines for comparison.

3.3 Multifield BM25: Analysis

From the experimental results, we can find that among all fields, `body` and `title` play fundamentally different but complementary roles. The `body` field provides broad coverage and recall, as it contains most of the document content, but it is also noisy—especially for long queries where many query terms may appear in irrelevant contexts. In contrast, the `title` field is short and highly concentrated, often explicitly summarizing the main topic of a document. Giving `title` a moderate weight helps the model focus on documents that are topically aligned with the query, improving early ranking quality without sacrificing too much recall (even in some times improve recall score). This explains why configurations that combine `body` and `title` with a clear emphasis on `body` (Exp-3.1b and Exp-3.2b) perform best overall. In these settings, the `body` field ensures coverage, while the `title` field acts as a strong precision signal that sharpens ranking at the top.

Besides these two fields, `url`, `keywords`, and `inlink` evidence introduces more nuanced behavior. These fields tend to be sparse and unevenly distributed across documents. When assigned small weights, they can occasionally provide useful signals. For example, URLs often contain structured hints such as "tables" or "schedule," and keywords can reflect author-provided summaries. However, these signals are far less reliable than `body` or `title`. As their weights increase, they begin to dominate the scoring function in ways that are not aligned with actual relevance, which leads to situations where documents are favored because they match surface-level metadata rather than substantive content. As we can see from the results, as the weight of the `body` field decreases and the weights of other fields increase, all metrics show a continuous decline. Even in Exp-3.1/3.2b, where the weight of the `title` field was boosted, it failed to compensate for the performance loss caused by the lack of matching on the substantive content. It also indicates the fundamental role of `body` while `title` is regarded as an optimization function for retrieval.

More specifically, when we pay attention to the failure cases (Exp-3.1/3.2e) in this experiment we will find that when `url` or `keywords` receive large weights, the model becomes overly sensitive to artifacts of document structure rather than meaning. This not only hurts precision but also significantly reduces recall, since many relevant documents simply do not contain rich metadata in these fields.

Finally, focus on the runtime, we find that combining more fields increases scoring complexity, especially for long queries, but the additional cost is only justified when the added fields meaningfully improve ranking quality. The best-performing configurations show that effective multifield BM25 is not about maximizing field coverage, but about prioritizing semantic reliability. A small number of strong fields with carefully weighted will get a perfect performance with less runtime cost.